

Adaptive Stability-Aware Scheduling for Cloud-Orchestrated Volunteer Computing Using Heterogeneous Desktop and Laptop Resources

Mishti Parikshit Gandhi Alan James
Abhyuday Thapliyal

Vellore Institute of Technology, Vellore, India

Abstract

Volunteer computing can provide substantial computational capacity by using idle desktop and laptop systems. However, unlike dedicated cloud resources, volunteer devices may become unavailable because of user activity, power changes, network interruptions, battery conditions, or correlated outages. This work presents an adaptive scheduling approach that continuously estimates the interruption risk of an already-running task and changes its execution strategy when the risk becomes significant. Instead of replicating every task from the beginning, the proposed system captures the current execution state and uses the checkpoint to start a secondary execution on a suitably independent volunteer resource. A cloud execution is used as a fallback when volunteer resources cannot provide sufficient protection.

The scheduler combines device stability, conditional survival probability, current system state, task progress, and residual execution time. Hedge selection additionally considers historical co-availability so that resources with correlated failure patterns are avoided. Experiments use a discrete-time simulation with heterogeneous desktop and laptop resources, correlated interruptions, different workload levels, and 300 random seeds per configuration. Under high instability, the proposed approach reduces wasted CPU time by 25.8% compared with independence-based replication while improving deadline completion from 95.56% to 97.38%. An ablation study shows that the live risk controller provides the largest improvement in resource efficiency.

Keywords: volunteer computing, adaptive scheduling, checkpointing, reliability, survival probability, replication, co-availability, cloud fallback

1 Introduction

Volunteer computing uses computational resources contributed by users to execute workloads that would otherwise require dedicated infrastructure. Desktop and laptop systems can provide useful computational capacity, but their availability is fundamentally different from that of managed servers. A user may start interacting with a machine, disconnect from the

network, switch from AC power to battery, shut down the system, or experience another temporary interruption while a task is executing.

This creates a scheduling problem that is different from conventional resource allocation. A task may initially be assigned to a suitable device and still become unsafe later in its execution. Therefore, reliability should not be treated as a static property known only at dispatch time.

A simple solution is to replicate tasks across multiple devices. However, starting multiple copies from the beginning wastes computation when the primary execution would have completed successfully. It also increases network, storage, and scheduling overhead. A more selective approach is to continue a task normally while its predicted interruption risk is low and introduce redundancy only when the risk increases.

The main idea of this work is therefore:

When a running volunteer task becomes sufficiently likely to be interrupted, capture its current state and start a redundant execution from that state on a resource whose availability is unlikely to be correlated with the primary resource.

The proposed scheduler combines four mechanisms:

1. continuous estimation of the interruption risk of active devices;
2. checkpoint-seeded hedging when the risk crosses a threshold;
3. co-availability-aware selection of the hedge resource; and
4. cloud execution when volunteer-based protection is insufficient.

The objective is not to maximize the number of replicas. Instead, the goal is to achieve deadline reliability while reducing wasted computation.

The main contributions are:

- a live interruption-risk model that is updated during execution;
- a risk-triggered checkpoint and hedge mechanism;
- a co-availability metric for avoiding correlated volunteer resources;
- a multi-level execution policy ranging from normal execution to cloud fallback; and
- an experimental evaluation under different instability levels, thresholds, and system sizes.

2 Related Work

Volunteer computing has traditionally focused on using large numbers of contributed machines to execute independent workloads. Systems such as BOINC demonstrate that substantial computational capacity can be obtained from distributed volunteer resources. However,

volunteer resources are subject to higher and less predictable availability variation than managed servers.

Checkpointing is a common mechanism for improving fault tolerance. Instead of restarting a failed computation, a checkpoint allows execution to resume from a previously saved state. The proposed method uses checkpointing in a different scheduling context: a checkpoint is also used as the starting point for a dynamically created redundant execution.

Replication and speculative execution can improve completion probability in unreliable distributed systems. Full replication, however, consumes additional resources even when failures do not occur. The proposed method limits replication to situations in which the estimated interruption risk justifies the additional execution.

Reliability modelling has also been studied using survival analysis. Kaplan–Meier estimation provides a non-parametric method for estimating survival probabilities from observed lifetimes. In this work, survival probability is used as an input to scheduling rather than as an isolated reliability statistic.

A further issue is correlated availability. Two devices that appear highly reliable individually may still be poor hedge candidates if they commonly become unavailable during the same periods. The proposed co-availability component therefore evaluates historical joint availability over the remaining task execution window.

3 System Model

The system consists of a scheduler, a set of heterogeneous volunteer devices, and an optional cloud execution service.

Each volunteer device is represented by:

$$D_i = (C_i, M_i, N_i, P_i, B_i, H_i)$$

where:

- C_i represents computational capacity,
- M_i represents available memory,
- N_i represents network capability,
- P_i represents the current power state,
- B_i represents battery-related state, and
- H_i represents historical availability information.

A task j has computational demand, memory demand, network requirements, an estimated execution time, and a deadline.

The scheduler maintains a continuously updated stability score:

$$SS_i = w_a A_i + w_u U_i + w_n N_i + w_h H_i$$

subject to

$$w_a + w_u + w_n + w_h = 1.$$

Here, A_i , U_i , N_i , and H_i represent availability, utilization, network, and historical stability factors.

Recent observations are incorporated using exponential updating:

$$X_i^{new} = \alpha X_i^{recent} + (1 - \alpha) X_i^{history}.$$

This prevents old observations from completely dominating current device conditions.

4 Adaptive Interruption-Risk Estimation

A central feature of the scheduler is that reliability is evaluated for the remaining execution period rather than only for the complete historical lifetime of a device.

Let T_i denote the time until interruption of device i . The conditional survival probability for a remaining period τ is:

$$P_i(\tau) = \Pr(T_i \geq \tau \mid \text{current session state}).$$

The corresponding interruption risk is:

$$R_i(\tau) = 1 - P_i(\tau).$$

The value of τ depends on the remaining expected execution time of the task. Thus, a device can have a good long-term historical reliability but still receive a high risk score when its current state indicates that it may soon become unavailable.

The scheduler considers information such as:

- current session age;
- historical availability;
- current power state;
- battery state;
- recent network behaviour;
- task progress; and
- remaining execution time.

Risk is re-evaluated during execution rather than being fixed at dispatch.

5 State-Aware Hazard Adjustment

Historical survival estimates cannot fully capture short-term changes in device behaviour. For example, a laptop operating on battery may have a different interruption likelihood from the same laptop operating on AC power.

The model therefore allows state-dependent hazard multipliers:

$$\lambda_i^{adj}(t) = \lambda_i(t) \prod_{k \in K_i} m_k,$$

where $\lambda_i(t)$ is the baseline hazard and m_k represents the effect of an observed state factor.

The adjusted survival probability can be expressed as:

$$P_i^{adj}(\tau) = \exp \left(- \int_{a_i}^{a_i + \tau} \lambda_i^{adj}(u) du \right).$$

The resulting risk is:

$$r_i = 1 - P_i^{adj}(\tau).$$

The implementation uses these state adjustments as part of the device ranking and dispatch decision.

6 Task-Device Suitability

Reliability alone is insufficient for scheduling. A highly stable device may still be unsuitable for a task because of insufficient CPU, memory, network capacity, or excessive execution time.

The suitability score for task j on device i is:

$$S_{ij} = w_c C_{ij} + w_m M_{ij} + w_n N_{ij} + w_s SS_i + w_p P_i(\tau_j) + w_d D_{ij} - w_k K_{ij}.$$

The terms represent computational capability, memory, network suitability, device stability, survival probability, deadline suitability, and estimated cost.

The scheduler first filters devices that cannot satisfy the basic task requirements and then ranks the remaining candidates.

7 Adaptive Execution Levels

The scheduler uses four execution levels:

- **L0 – Normal execution:** the task runs on one volunteer device.
- **L1 – Checkpointed execution:** the task continues normally but its state is periodically protected.

- **L2 – Checkpoint-seeded hedging:** a secondary execution is started from the latest checkpoint.
- **L3 – Cloud fallback:** the task is continued on cloud infrastructure when volunteer resources do not provide adequate protection.

Using thresholds R_1 , R_2 , and R_3 :

$$\text{Level} = \begin{cases} L0, & R < R_1, \\ L1, & R_1 \leq R < R_2, \\ L2, & R_2 \leq R < R_3, \\ L3, & R \geq R_3. \end{cases}$$

This allows redundancy to be introduced progressively instead of treating all tasks identically.

8 Live Risk Controller

The most important difference between the proposed scheduler and a static replication strategy is that an already-running task can change execution level.

For an active task, the scheduler periodically performs:

1. collect current device state;
2. estimate remaining task execution time;
3. update interruption risk;
4. compare the risk against execution thresholds;
5. create a checkpoint when required;
6. select a hedge candidate if redundancy is required; and
7. cancel unnecessary replicas after successful completion.

This means that redundancy is triggered by the changing state of the running task rather than being decided entirely at initial dispatch.

9 Checkpoint-Seeded Hedging

Traditional replication may launch several copies from the beginning of a task. The proposed approach instead uses the most recent checkpoint.

Suppose a task has completed fraction q of its computation when the primary device becomes risky. A checkpoint captures the current state and allows another device to begin at approximately the same progress point.

The two executions can then proceed in parallel:

Algorithm 1 Live Risk Control

Require: Active task j , primary device i

```
1: Observe current device state
2: Estimate remaining execution time  $\tau$ 
3: Compute  $R_i(\tau)$ 
4: if  $R_i(\tau) < R_1$  then
5:   Continue normal execution
6: else if  $R_i(\tau) < R_2$  then
7:   Maintain checkpoint protection
8: else if  $R_i(\tau) < R_3$  then
9:   Create checkpoint
10:  Select independent hedge device
11:  Start hedge from checkpoint
12: else
13:  Create checkpoint
14:  Attempt hedge placement
15:  if no suitable volunteer hedge exists then
16:    Start cloud execution from checkpoint
17:  end if
18: end if
```

Primary: $q \rightarrow 1$

Hedge: $q \rightarrow 1$.

The first execution to produce a valid result is accepted. Remaining executions are terminated where possible.

This reduces the amount of duplicated computation because work already performed before the hedge was created does not need to be repeated.

A simplified checkpoint contains:

- task identifier;
- checkpoint sequence number;
- completed-work indicator;
- application state;
- integrity information; and
- timestamp.

The checkpoint size and transfer cost are included in the simulation as illustrative parameters.

10 Co-Availability-Aware Hedge Selection

Selecting the most reliable device individually does not necessarily produce the most reliable replica pair. Two devices may have similar interruption patterns because they belong to the same physical environment, network, power group, or user schedule.

The proposed method therefore stores historical availability as a time bitmap. For a time slot w :

$$x_i(w) = \begin{cases} 1, & \text{device available,} \\ 0, & \text{device unavailable.} \end{cases}$$

For a candidate set R , joint availability is determined by:

$$x_R(w) = \bigwedge_{i \in R} x_i(w).$$

The empirical probability that all devices in the set become unavailable within a sampled window is:

$$F_{emp}(R) = \frac{1}{N} \sum_{w=1}^N \prod_{i \in R} (1 - x_i(w)).$$

For comparison, assuming independence gives:

$$F_{ind}(R) = \prod_{i \in R} (1 - p_i),$$

where p_i is the individual interruption probability.

A shrinkage estimate combines the two:

$$F(R) = \frac{NF_{emp}(R) + \alpha_s F_{ind}(R)}{N + \alpha_s}.$$

A candidate hedge k is selected by minimizing:

$$F(R \cup \{k\}) + \lambda Cost_k.$$

Thus, the scheduler considers both the reliability of the candidate and its relationship with the resources already executing the task.

11 Cloud Fallback

Volunteer resources may occasionally be unable to provide sufficient reliability. In this case, the scheduler can transfer execution to cloud infrastructure.

Algorithm 2 Co-Availability-Aware Hedge Selection

Require: Primary set R , candidate devices C

- 1: Remove candidates that fail capability requirements
 - 2: **for all** $k \in C$ **do**
 - 3: Estimate $F(R \cup \{k\})$
 - 4: Compute candidate cost
 - 5: Compute combined objective
 - 6: **end for**
 - 7: Select candidate with minimum objective
 - 8: **return** selected device
-

The cloud is treated as a fallback rather than the default execution environment. This preserves the cost advantage of volunteer computing under normal conditions while providing a stronger recovery mechanism when the available volunteer population becomes unsuitable.

The cloud execution can also begin from the latest checkpoint:

Volunteer state $\rightarrow$ Checkpoint $\rightarrow$ Cloud execution.

This avoids restarting the complete computation.

12 Overall Scheduling Procedure

The complete scheduling lifecycle is:

1. collect device capability and availability information;
2. estimate the stability of each device;
3. select a suitable primary device;
4. dispatch the task;
5. periodically update the interruption risk;
6. checkpoint when the risk enters the protection region;
7. launch a hedge when the risk becomes sufficiently high;
8. choose the hedge using co-availability information;
9. use cloud fallback if necessary;
10. accept the first valid completed result; and
11. terminate unnecessary executions.

This lifecycle is designed to balance three competing objectives:

Reliability $\leftrightarrow$ Wasted computation $\leftrightarrow$ Cloud cost.

Increasing redundancy can improve deadline completion but may also increase resource consumption. The adaptive controller attempts to introduce redundancy only when it provides sufficient expected benefit.

13 Experimental Methodology

The proposed scheduler was evaluated using a discrete-time simulation.

Each simulation step represents five minutes. Four weeks of simulated history are generated for estimating availability behaviour, followed by a fresh evaluation week.

The simulated volunteer population contains heterogeneous desktop and laptop devices with:

- 2, 3, or 5 CPU cores;
- 4, 8, or 16 GB RAM;
- 1, 10, or 100 Mbps uplink capacity;
- desktop or laptop characteristics; and
- AC or battery operating states.

Individual interruptions and correlated group outages are included. Laptop availability is affected by battery and power conditions.

Three workload instability conditions are evaluated:

- **Stable:** relatively reliable volunteer availability;
- **Moderate:** increased interruption frequency; and
- **High:** substantially unstable volunteer availability.

Task arrivals follow a Poisson-like process. Task deadlines are selected between approximately 1.6 and 3.0 times the expected execution time.

Each main configuration uses 300 independent random seeds. Reported uncertainty is a 95% confidence interval computed using:

$$\bar{x} \pm 1.96 \frac{s}{\sqrt{n}}.$$

The principal metrics are:

- deadline completion reliability;
- wasted CPU time;
- checkpoint storage;
- cloud execution consumption; and
- replica launches per task.

The experiments model independent tasks rather than DAG workflows.

14 Main Results

Table 1 compares four scheduling strategies.

Table 1: Main simulation results. Values are mean $\pm$ 95% CI.

Condition	Method	Deadline (%)	Wasted CPU	Cloud (inst-sec)
Stable	Capability only	99.86 ± 0.04	18138 ± 1303	–
	Capability + Stability	99.88 ± 0.04	16878 ± 1204	–
	Independence replication	99.84 ± 0.04	11442 ± 790	–
	Proposed	98.84 ± 0.27	10984 ± 746	4140
Moderate	Capability only	99.33 ± 0.12	41065 ± 2248	–
	Capability + Stability	99.27 ± 0.12	39657 ± 2144	–
	Independence replication	99.49 ± 0.09	22187 ± 1104	–
	Proposed	98.36 ± 0.31	19189 ± 894	16274
High	Capability only	95.56 ± 0.49	86938 ± 4231	–
	Capability + Stability	95.64 ± 0.49	83373 ± 4224	–
	Independence replication	97.18 ± 0.42	41174 ± 1739	–
	Proposed	97.38 ± 0.34	30546 ± 1067	60561

Under stable conditions, the proposed method provides slightly lower deadline completion than the simpler alternatives because it introduces selective protection and cloud fallback only when required.

The main benefit appears under high instability. Compared with independence-based replication, wasted CPU decreases from 41,174 to 30,546 units:

$$\frac{41174 - 30546}{41174} \times 100 \approx 25.8\%.$$

At the same time, deadline completion increases from 97.18% to 97.38%.

For moderate instability, wasted CPU decreases from 22,187 to 19,189:

$$\frac{22187 - 19189}{22187} \times 100 \approx 13.5\%.$$

Under stable conditions, the reduction is approximately:

$$\frac{11442 - 10984}{11442} \times 100 \approx 4.0\%.$$

These results indicate that adaptive protection becomes increasingly useful as volunteer instability increases.

15 Ablation Study

The high-instability configuration was used to evaluate the contribution of individual components.

Table 2: Ablation study under high instability.

Configuration	Deadline (%)	Wasted CPU	Cloud
Capability only	95.56 ± 0.49	86938 ± 4231	–
+ Stability	95.64 ± 0.49	83373 ± 4224	–
+ Conditional survival	95.38 ± 0.49	81478 ± 4202	–
+ Dispatch levels	95.48 ± 0.49	79679 ± 4234	–
+ Live risk controller	98.50 ± 0.23	33184 ± 1152	51883
+ Hazard multipliers	98.75 ± 0.21	30630 ± 1068	60699
+ Co-availability	98.75 ± 0.21	30538 ± 1068	60561
Full lifecycle	97.38 ± 0.34	30546 ± 1067	60561

The largest change occurs when the live risk controller is introduced. Compared with the dispatch-only configuration, wasted CPU falls from 79,679 to 33,184:

$$\frac{79679 - 33184}{79679} \times 100 \approx 58.4\%.$$

This supports the main design argument that re-evaluating active tasks is more important than simply improving the initial device ranking.

Hazard multipliers provide an additional reduction, while the isolated co-availability contribution is smaller in the main system experiment.

16 Co-Availability Evaluation

A system-level experiment used 40 devices and 150 tasks over 300 seeds.

Table 3: System-level co-availability comparison.

Method	Wasted CPU	Reliability (%)	Replica-hours/task
Independence hedge choice	29945 ± 1069	98.71	1.192
Co-availability	29839 ± 1065	98.72	1.191

The difference is small at this scale because many candidate devices have similar availability histories.

A separate isolated placement experiment was therefore performed using 500 independently sampled correlated device configurations. Each configuration contained 28 devices and four correlated groups.

Table 4: Correlated-placement experiment.

Method	Mean replicas	Joint interruption (%)	Runs exceeding target (%)
Independence ranking	3.99	15.85	73.1
Co-availability	3.52	6.93	51.1

The isolated experiment demonstrates the importance of considering correlation. Independence-based ranking requires more replicas while still producing substantially higher joint interruption probability.

Neither method always reaches the strict target of 0.02 because the number of available replicas and the underlying correlation structure limit the achievable reliability.

17 Threshold Sensitivity

The threshold controlling the transition into active hedging was varied while maintaining $R_1 = 0.05$.

Table 5: Sensitivity to risk thresholds.

R_2	R_3	Wasted CPU	Deadline (%)
0.10	0.40	28251 ± 945	97.77 ± 0.31
0.20	0.50	29354 ± 1000	97.58 ± 0.32
0.30	0.60	29972 ± 1034	97.41 ± 0.34
0.40	0.70	30471 ± 1059	97.33 ± 0.34

Lower hedging thresholds provide more aggressive protection and consequently higher cloud consumption. Higher thresholds reduce cloud use but slightly increase wasted CPU and reduce deadline completion.

The results therefore show a trade-off rather than a single universally optimal threshold.

18 Scaling with Volunteer Population

The proposed scheduler was also evaluated under high instability while varying the number of available devices.

Table 6: Scaling results with 150 tasks.

Devices	Deadline (%)	Wasted CPU	Cloud
10	76.80 ± 1.43	31122 ± 1190	102701 ± 6546
20	94.40 ± 0.62	31961 ± 1067	82792 ± 5492
30	97.38 ± 0.34	30546 ± 1067	60561 ± 4817
40	98.00 ± 0.25	29839 ± 1065	41213 ± 3361
50	98.29 ± 0.27	28718 ± 1048	28249 ± 3212

Increasing the volunteer population improves deadline completion and reduces dependence on cloud execution. Replica launches per task also decrease from approximately 1.38 with 10 devices to approximately 1.21 with 50 devices.

This behaviour is expected because a larger resource pool provides more opportunities to find a suitable independent hedge.

19 Discussion

The experiments support three main observations.

First, static scheduling is insufficient when device reliability changes during execution. The large reduction in wasted CPU after introducing the live risk controller demonstrates the value of reacting to current conditions.

Second, starting replicas from checkpoints is more resource-efficient than replicating every task from the beginning. The scheduler spends additional resources primarily when the predicted risk becomes significant.

Third, the usefulness of co-availability depends on the correlation structure of the resource pool. In the main system experiment its effect is modest, but the isolated correlated-placement experiment shows a much larger difference. This suggests that co-availability becomes particularly important when volunteer resources share common failure causes.

The cloud fallback introduces additional cost, but it is used selectively. The scaling results show that this dependence decreases as the volunteer population becomes larger.

20 Limitations

The evaluation is simulation-based rather than a deployment on a large real-world volunteer network. The device availability model is synthetic and includes simplified individual and correlated interruption behaviour.

Checkpoint sizes, transfer costs, and cloud execution costs are represented using illustrative parameters rather than measurements from a production application.

The current experiments consider independent tasks rather than complex DAG workflows. The lifecycle verification model is also simplified and does not represent every production concern such as complete result canonicalization, canary execution, or long-term feedback control.

Therefore, the reported values should be interpreted as comparative simulation results rather than direct predictions of production performance.

21 Future Work

Future work can evaluate the scheduler using real volunteer-computing traces and heterogeneous physical devices. Additional work can investigate more detailed power and network models, larger correlated resource populations, and workflow-level scheduling.

The risk controller could also be extended with online learning so that state-transition behaviour is learned continuously from observed execution outcomes.

22 Conclusion

This paper presented an adaptive stability-aware scheduler for volunteer computing using heterogeneous desktop and laptop resources. The central approach is to monitor the interruption risk of running tasks and introduce redundancy only when the predicted risk becomes significant.

Rather than restarting a complete task, the scheduler uses a checkpoint to seed a secondary execution. The secondary resource is selected using both individual reliability and historical co-availability, reducing the probability of correlated interruption. When volunteer resources are insufficient, cloud execution provides a fallback.

Simulation results show that the benefit of adaptive scheduling becomes stronger as volunteer instability increases. Under high instability, the proposed method reduces wasted CPU by approximately 25.8% compared with independence-based replication while achieving slightly higher deadline completion. The ablation study further shows that the live risk controller is the most influential component for reducing wasted computation.

Overall, the results indicate that reliability-aware scheduling should be treated as a continuous process rather than a one-time decision made when a task is initially dispatched.

References

- [1] D. P. Anderson, “BOINC: A System for Public-Resource Computing and Storage,” Proceedings of the 5th IEEE/ACM International Workshop on Grid Computing, 2004.
- [2] E. Korpela, D. Werthimer, D. Anderson, J. Cobb, and M. Lebofsky, “SETI@home—Massively Distributed Computing for SETI,” Computing in Science & Engineering, vol. 3, no. 1, 2001.
- [3] D. P. Anderson, “Public Computing: Reconnecting People to Science,” Proceedings of the Conference on Shared Knowledge and the Web, 2002.
- [4] L. Young, “On the Computation of Survival Rates,” Technometrics, vol. 16, no. 3, 1974.
- [5] E. L. Kaplan and P. Meier, “Nonparametric Estimation from Incomplete Observations,” Journal of the American Statistical Association, vol. 53, no. 282, pp. 457–481, 1958.
- [6] E. N. Elnozahy, L. Alvisi, Y.-M. Wang, and D. B. Johnson, “A Survey of Rollback-Recovery Protocols in Message-Passing Systems,” ACM Computing Surveys, vol. 34, no. 3, pp. 375–408, 2002.
- [7] K. M. Chandy and L. Lamport, “Distributed Snapshots: Determining Global States of Distributed Systems,” ACM Transactions on Computer Systems, vol. 3, no. 1, pp. 63–75, 1985.
- [8] L. A. Barroso, U. Holzle, and P. Ranganathan, *The Datacenter as a Computer: Designing Warehouse-Scale Machines*, Morgan & Claypool, 2018.
- [9] J. Dean and L. A. Barroso, “The Tail at Scale,” Communications of the ACM, vol. 56, no. 2, pp. 74–80, 2013.
- [10] E. Gelenbe and R. M. Lent, “Power-Aware Performance in Computing Systems,” Future Generation Computer Systems, vol. 18, no. 6, pp. 755–766, 2002.
- [11] F. Cappello, “Fault Tolerance in Petascale/Exascale Systems: Current Knowledge, Challenges and Research Opportunities,” International Journal of High Performance Computing Applications, vol. 23, no. 3, pp. 212–226, 2009.

- [12] S. Chandra and P. M. Chen, “How Failures Are Handled in Large Distributed Systems,” *Communications of the ACM*, vol. 50, no. 11, pp. 70–75, 2007.
- [13] K. V. Vishwanath and N. Nagappan, “Characterizing Cloud Computing Hardware Reliability,” *Proceedings of the 1st ACM Symposium on Cloud Computing*, 2010.
- [14] K. P. Birman, *Reliable Distributed Systems: Technologies, Web Services, and Applications*, Springer, 2005.
- [15] A. S. Tanenbaum and M. van Steen, *Distributed Systems: Principles and Paradigms*, Pearson, 2017.
- [16] I. Foster, “The Anatomy of the Grid: Enabling Scalable Virtual Organizations,” *International Journal of High Performance Computing Applications*, vol. 15, no. 3, pp. 200–222, 2001.
- [17] R. Buyya, C. S. Yeo, S. Venugopal, J. Broberg, and I. Brandic, “Cloud Computing and Emerging IT Platforms: Vision, Hype, and Reality for Delivering Computing as the 5th Utility,” *Future Generation Computer Systems*, vol. 25, no. 6, pp. 599–616, 2009.
- [18] M. Armbrust et al., “A View of Cloud Computing,” *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [19] C. Reiss, J. Wilkes, and J. Hellerstein, “Google Cluster-Usage Traces: Format + Schema,” *Google Technical Report*, 2011.
- [20] M. Zaharia et al., “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing,” *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation*, 2012.